
Servicios de red

Manuel C. Piñeiro Mourazos

2026-05-04

Índice

1	Servicios de red	2
1.1	Correo electrónico SMTP - Envío de emails	2
1.1.1	Comunicación SMTP	3
1.2	Servicios HTTP	6
1.2.1	API REST	7
1.2.2	API RESTful	8
2	Paquete net.smtp	9
2.1	Ejemplo de uso de net/smtp	9
2.2	Envío de emails usando net/smtp con el servidor SMTP de Gmail . .	12

1. Servicios de red

Cuando hablamos de servicios de red nos referimos a las distintas aplicaciones que implementan protocolos como SSH, SMTP, DNS, Telnet, FTP, etc. asociados a los respectivos servicios de conexiones seguras, correo electrónico, resolución de nombres, etc. Estos servicios se encuentran en toda la red y proporcionan acceso interactivo remoto, transferencia de archivos y funciones críticas de administración para el funcionamiento de distintos sistemas.

En este tema vamos a ver dos de estos servicios de red:

- El servicio correo electrónico a través del protocolo SMTP. Enviaremos correos utilizando este protocolo y distintas librerías y servicios.
- El servicio web, mediante el protocolo HTTP, este último en el contexto de la creación de una API RESTful utilizando el framework GIN.

1.1. Correo electrónico SMTP - Envío de emails

SMTP (Simple Mail Transfer Protocol) es un protocolo de comunicación utilizado para el intercambio de mensajes de correo electrónico entre distintos dispositivos a través de Internet. Este protocolo está construido sobre TCP, protocolo de la capa de transporte, y se encuadra a su vez en la capa de aplicación.

Este protocolo, debido a sus limitaciones, se encarga de la tarea de enviar correos entre el cliente y el servidor de correo y entre servidores, mientras que otros protocolos como IMAP (Internet Message Access Protocol) o POP3 (Post Office Protocol version 3) se encargan de la tarea de recibir y descargar los correos electrónicos desde el servidor al cliente.

Como el resto de protocolos que veremos en esta unidad, usa por debajo el protocolo de transporte TCP, concretamente el puerto 25 para la comunicación entre servidores de correo y el puerto 587 para la comunicación entre clientes de correo y servidores de correo.

Este protocolo usa un modelo de comunicación cliente-servidor, donde el cliente de correo (como Outlook, Thunderbird o una aplicación personalizada) se conecta al servidor de correo (como Gmail, Yahoo Mail o un servidor de correo empresarial) para enviar mensajes. El proceso de envío de correo electrónico a través de SMTP implica varios pasos, como la autenticación del cliente, la construcción del mensaje, la transmisión del mensaje al servidor y la entrega del mensaje al destinatario.

1.1.1. Comunicación SMTP

En el siguiente gráfico se muestra un ejemplo del uso del protocolo SMTP y su relación con IMAP y POP3 para el envío y recepción de correos electrónicos:

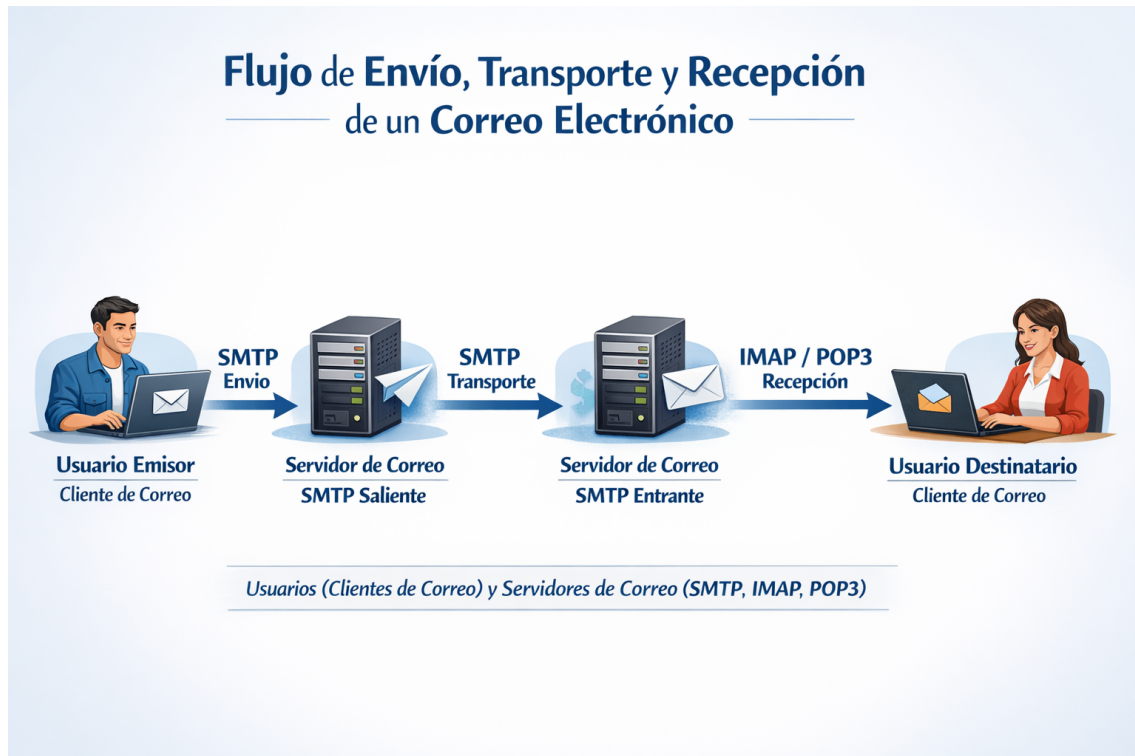


Figura 1: Diagrama de transmisión de un correo electrónico utilizando SMTP, IMAP y POP3

En el gráfico (*generado por copilot/dall-e*) se pueden observar los siguientes procesos:

- **Generación y Envío (SMTP):** El proceso comienza cuando el remitente redacta el mensaje en su programa cliente y lo envía. Para esta fase de salida desde el cliente hacia el servidor de correo saliente, **se utiliza el protocolo SMTP**.
- **Transmisión entre Servidores (SMTP):** El servidor del remitente localiza el servidor del destinatario a través de Internet y le transfiere el mensaje. Esta comunicación de servidor a servidor también se realiza mediante SMTP.
- **Recepción y Almacenamiento:** El servidor del destinatario recibe el correo y lo guarda en el buzón correspondiente.
- **Descarga por el Destinatario (IMAP/POP3):** Finalmente, el destinatario abre su programa cliente para leer el mensaje. Para descargar o sincronizar el correo desde el servidor, se utilizan los protocolos IMAP (que mantiene los correos sincronizados en el servidor) o POP3 (que suele descargarlos y eliminarlos del servidor).

La comunicación entre dos servidores de correo a través de SMTP se realiza mediante una serie de comandos y respuestas que permiten la transferencia de mensajes entre un cliente y un servidor (o entre dos servidores). Estos comandos son las “instrucciones” que indican qué acción se debe realizar en cada paso del envío.

Recordemos que esto realmente se realiza a través de una conexión TCP establecida entre el cliente y el servidor, y cada comando se envía como una línea de texto seguida de un retorno de carro y un salto de línea (CRLF). El servidor responde a cada comando con un código de estado que indica si la operación fue exitosa o si hubo algún error. Todo, últimamente, se reduce a establecer una conexión TCP y el intercambio de secuencias de comandos y respuestas en forma de secuencias de bytes.

Algunos de los comandos SMTP más comunes incluyen:

Comando

SMTP Función Principal

HELO Inicia la sesión. Es el “hola” del cliente para presentarse al servidor. **EHLO** / **EH-** (Extended HELO) es la versión moderna que informa al servidor que el **LO** cliente soporta extensiones adicionales (ESMTP).

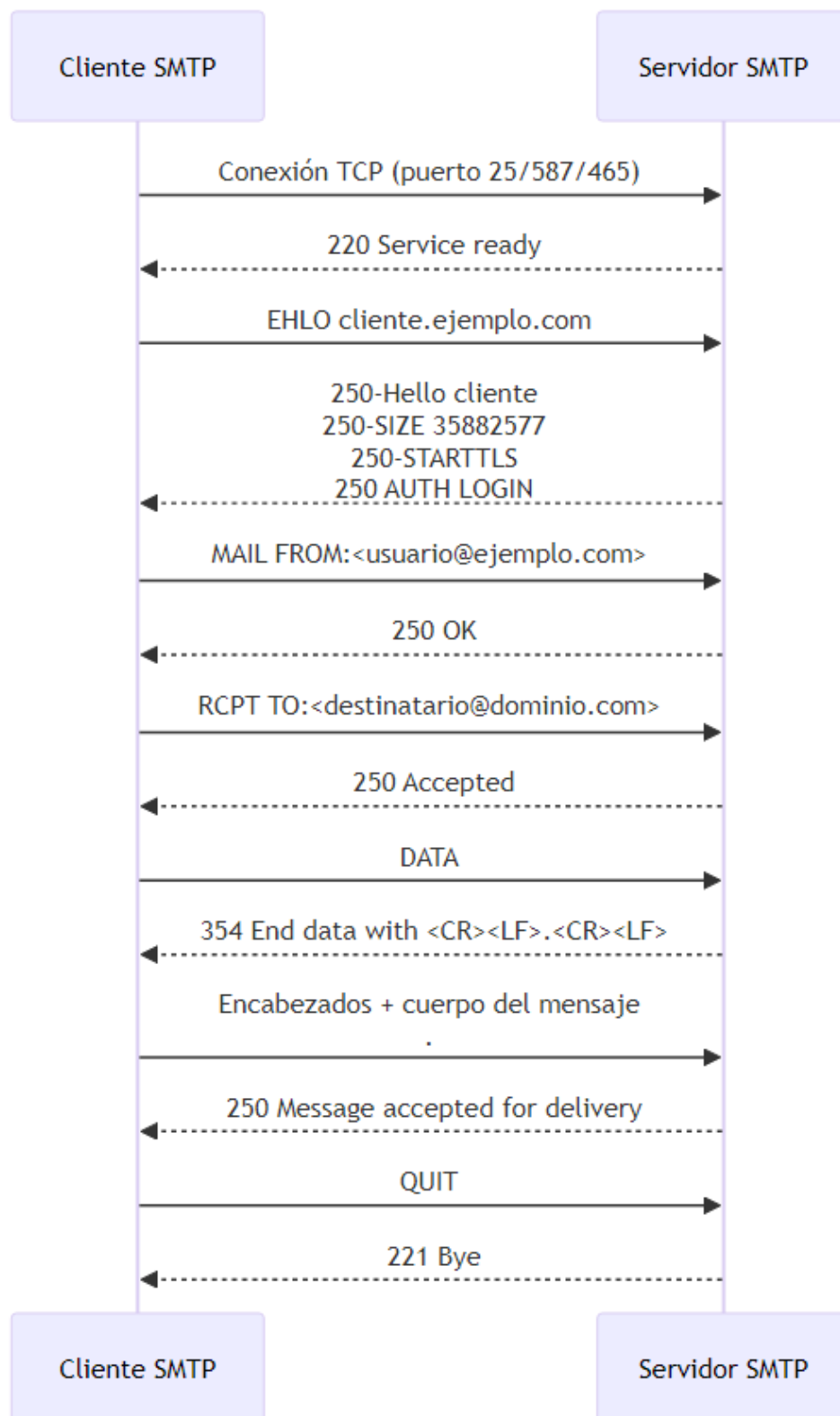
MAIL Inicia formalmente la transacción del mensaje. Especifica la dirección de **FROM:** correo electrónico del remitente (quién envía el correo).

RCPT Especifica la dirección de correo electrónico del destinatario. Si el correo **TO:** va dirigido a varias personas, este comando se envía varias veces, una por cada destinatario.

DATA Indica al servidor que los datos que vienen a continuación son el contenido real del mensaje. Las cabeceras como “Asunto” o “Fecha”, y el cuerpo del correo.

QUIT Pide al servidor que cierre la conexión. Finaliza la sesión SMTP de forma ordenada una vez que los correos han sido enviados.

Un ejemplo de comunicación SMTP entre un cliente y un servidor podría ser el siguiente:



1.1.1.1. Puertos de SMTP Los puertos que se utilizan para la comunicación SMTP son cuatro, cada uno asociado a un tipo de cifrado diferente para el envío

de correos electrónicos:

- 25: Este puerto se utiliza para la comunicación entre servidores de correo, y es el puerto estándar para el envío de correos electrónicos. Sin embargo, debido a que no requiere cifrado, muchos proveedores de servicios de Internet bloquean este puerto para evitar el envío de correos electrónicos no deseados o maliciosos.
- 2525: Este puerto es una alternativa al puerto 25 pero, diferencia de éste, el puerto 2525 puede ser cifrado utilizando TLS (Transport Layer Security), lo que proporciona una capa adicional de seguridad para la transmisión de correos electrónicos.
- 587: Este puerto es el puerto registrado por la IANA (Internet Assigned Numbers Authority) como el puerto seguro para SMTP. Requiere una conexión TLS explícita, lo que significa que el cliente debe iniciar la conexión y luego negociar el cifrado con el servidor. Sin embargo, si el servidor no admite TLS, el mensaje se enviará en texto plano, lo que representa, de nuevo, un riesgo de seguridad.
- 465: Este puerto se utiliza para SMTP sobre SSL (Secure Sockets Layer) de forma implícita. Esto significa que la conexión se establece directamente con SSL, y si el servidor no admite esta configuración, la operación se abortará.

Terminamos aquí con los aspectos teóricos relacionados con el protocolo SMTP y el envío de correos electrónicos. Veremos ejemplos prácticos utilizando distintas librerías y servicios para enviar correos electrónicos en la parte práctica de esta unidad.

1.2. Servicios HTTP

El siguiente servicio que veremos será el servicio basado en el protocolo HTTP (Hypertext Transfer Protocol). Nos referimos a los servicios web, que son aplicaciones que se ejecutan en un servidor y que pueden ser accedidos a través de la red utilizando el protocolo HTTP. Este protocolo ofrece un mecanismo para la comunicación entre clientes y servidores en la web, permitiendo la transferencia de datos y la interacción con recursos a través de URLs (Uniform Resource Locators). HTTP es un protocolo de texto plano que se basa en un modelo de solicitud-respuesta (*HTTP requests* y *HTTP responses*), donde los clientes envían solicitudes al servidor y el servidor responde con los datos solicitados o con información sobre el resultado de la solicitud. HTTP se encuentra en la capa de aplicación y se implementa (al igual que SMTP) sobre el protocolo de transporte TCP, utilizando el puerto 80 para HTTP y el puerto 443 para HTTPS (HTTP seguro con cifrado SSL/TLS).

Cuando hablamos de que se trata de que sigue el modelo cliente-servidor, nos referimos a que el cliente (como un navegador web o una aplicación móvil) realiza

solicitudes HTTP al servidor (que aloja la aplicación web o API) para acceder a recursos. Algunos ejemplos de clientes serían un navegador web que accede a una página web, una aplicación móvil que consume una API RESTful o un programa de línea de comandos que interactúa con un servicio web como **curl**. Algunos ejemplos de servidores serían un servidor web como Apache o Nginx que aloja un sitio web, un servidor de aplicaciones que ejecuta una API RESTful o un servidor de correo electrónico que proporciona acceso a través de HTTP.

1.2.1. API REST

Empecemos por aclarar en que consiste una API. Una API (Application Programming Interface) es un conjunto de reglas y protocolos que permiten a diferentes aplicaciones o sistemas comunicarse entre sí. En el contexto de los servicios web, una API define **cómo los clientes pueden interactuar con un servidor** para acceder a recursos o **realizar acciones** específicas. Las API pueden ser utilizadas para exponer funcionalidades de una aplicación a otros desarrolladores, permitiendo la integración de servicios y la creación de aplicaciones más complejas.

A su vez, una API REST (Representational State Transfer) es un tipo de API que sigue los principios de REST. REST se basa en principios como la separación de recursos, la utilización de métodos HTTP y la representación de recursos en formatos como JSON o XML.

1.2.1.1. Principios de REST Los principios fundamentales que debe cumplir una API RESTful son los siguientes:

1. *Resource-Based*: En REST, los recursos (también vistos como objetos) son los elementos clave. Cada recurso se identifica mediante una URL única y se representa en un formato específico (como JSON o XML). Por ejemplo, en una API de gestión de usuarios, un recurso podría ser un usuario específico identificado por su ID, como `https://api.com/usuarios/123`.
2. *State-Less*: Las API RESTful son sin estado, lo que significa que cada el servidor no guarda en memoria las peticiones anteriores de un cliente. La solicitud del cliente al servidor debe contener toda la información necesaria para entender y procesar la solicitud. El servidor no debe almacenar ningún estado de la sesión entre solicitudes. Esto permite que las API RESTful sean escalables y fáciles de mantener.
3. *Uso de métodos HTTP*: REST utiliza los métodos HTTP para realizar operaciones sobre los recursos. Estos métodos implementan el conjunto de operaciones CRUD (Create, Read, Update, Delete) que permiten crear, leer, actualizar y eliminar recursos. Cada método HTTP tiene un propósito específico y se uti-

liza para realizar una acción particular sobre un recurso . Los métodos más comunes son:

- GET: Se utiliza para recuperar información de un recurso.
- POST: Se utiliza para crear un nuevo recurso.
- PUT: Se utiliza para actualizar un recurso existente.
- DELETE: Se utiliza para eliminar un recurso.

4. Representación de recursos: Como mencionamos en el punto 1, los recursos en una API RESTful se representan en formatos como JSON o XML. Esto permite que los clientes puedan interpretar y utilizar los datos de manera eficiente. Por ejemplo, una respuesta a una solicitud GET para un recurso de usuario podría devolver un objeto JSON con la información del usuario, como su nombre, correo electrónico y fecha de creación. El cliente será el encargado de interpretar esta representación y utilizarla según sus necesidades (crear un objeto en memoria, mostrar la información al usuario, etc.).

REST vs RESTful: Rest hace referencia a un estilo de arquitectura de software para diseñar servicios web (los cuatro principios que acabamos de ver), mientras que RESTful se refiere a una API que sigue los principios de REST (la implementación). En otras palabras, REST es el conjunto de principios y RESTful es la implementación concreta de esos principios en una API.

1.2.2. API RESTful

Una API (Application Programming Interface) RESTful es un tipo de API que sigue los principios de REST (Representational State Transfer).

Para utilizar una API RESTful, los clientes realizan solicitudes HTTP a un servidor que expone la API (*http requests*), y el servidor responde con los datos solicitados (*http responses*) o realiza las acciones correspondientes. Dependiendo del método HTTP (GET, POST, DELETE, PUT, etc.) y la URL de la solicitud (el campo *target* de la primera línea de una petición http), el servidor puede realizar diferentes operaciones sobre los recursos que maneja la API.

La API responderá mediante un HTTP response que incluirá un código de estado HTTP (como 200 OK, 404 Not Found, 500 Internal Server Error, etc.) y, en caso de éxito, el cuerpo de la respuesta contendrá los datos solicitados o una confirmación de la acción realizada, generalmente en formato JSON o XML.

Aquí finalizamos con la parte teórica relacionada con el protocolo HTTP y los servicios web, así como con las API RESTful. En la parte práctica de esta unidad veremos cómo crear una API RESTful utilizando el framework GIN en Go, y cómo consumir esta API desde un cliente HTTP utilizando distintas herramientas y librerías.

2. Paquete `net.smtp`

El paquete `net/smtp` es una biblioteca estándar de Go que proporciona funciones para enviar correos electrónicos utilizando el protocolo SMTP. Permite a los desarrolladores enviar correos electrónicos desde sus aplicaciones Go de manera sencilla y eficiente. El paquete `net/smtp` ofrece una interfaz simple para configurar el servidor SMTP, autenticar usuarios y enviar mensajes de correo electrónico con diferentes formatos, como texto plano o HTML.

En los ejemplos de uso de servidores de email veréis que la mayoría dispone de sus propias bibliotecas para enviar correos electrónicos, como en el caso de `jetmail` o `mailersend`.

2.1. Ejemplo de uso de `net/smtp`

En este ejemplo veremos como enviar un email usando el servidor SMTP de Google (host: `smtp.gmail.com`) utilizando el paquete `net/smtp` de Go. Para ello, necesitaremos configurar el servidor SMTP de Gmail, autenticar nuestra cuenta de Gmail y luego enviar un correo electrónico a un destinatario específico.

Para poder utilizar Gmail para enviar un correo electrónico hemos de obtener una **app password** para luego utilizarla para autenticar nuestra cuenta de Gmail en el servidor SMTP, y así poder enviar correos electrónicos desde nuestra aplicación Go. A continuación, se detallan los pasos para generar una contraseña de aplicación y configurar el servidor SMTP de Gmail.

2.1.0.1. Generar la contraseña de aplicación Una contraseña de aplicación es una contraseña específica que se utiliza para autenticar aplicaciones de terceros con tu cuenta de Google. Es un método prácticamente obsoleto, ya que Google ha deshabilitado el acceso a aplicaciones menos seguras, pero aún es posible generar una contraseña de aplicación para ciertas aplicaciones que no admiten la autenticación de dos factores. Este es nuestro caso con `net/smtp`.

Para generar una contraseña de aplicación, seguiremos los siguientes pasos:

1. Hemos de acceder a la página de seguridad de tu cuenta de Google: <https://myaccount.google.com/security>
2. En la sección “Iniciar sesión en Google”, haz clic en “Contraseñas de aplicaciones”.
3. Si se te solicita, ingresa tu contraseña de Google para verificar tu identidad.
4. Finalmente, selecciona la aplicación para la que deseas generar la contraseña de aplicación (en este caso, “Correo”) y el dispositivo para el que deseas gene-

rar la contraseña (puedes seleccionar “Otro” y escribir un nombre personalizado). Luego, haz clic en “Generar” para obtener la contraseña de aplicación.

Ten en cuenta que has de copiar la contraseña de aplicación generada, ya que no podrás volver a verla después de cerrar la ventana. Esta contraseña de aplicación es la que utilizarás para autenticar tu cuenta de Gmail en el servidor SMTP y enviar correos electrónicos desde tu aplicación Go.

Si no aparece la opción “Contraseñas de aplicaciones”, es posible que tu cuenta de Google no tenga habilitada la autenticación de dos factores, lo que es necesario para generar contraseñas de aplicación. En ese caso, deberás habilitar la autenticación de dos factores en tu cuenta de Google antes de poder generar una contraseña de aplicación.

Si, teniendo activada la autenticación de dos factores, no aparece la opción “Contraseñas de aplicaciones”, se puede acceder directamente a la página de generación de contraseñas de aplicación a través del siguiente enlace: <https://myaccount.google.com/apppasswords> En algunas cuentas, aún teniendo activada la autenticación de dos factores, no aparece la opción “Contraseñas de aplicaciones”. En ese caso, se puede acceder directamente a la página de generación de contraseñas de aplicación a través del siguiente enlace: <https://myaccount.google.com/apppasswords>

Una vez obtenida la contraseña no está de más recordar que, para mantener la seguridad de tu cuenta de Google, es importante no compartir esta contraseña de aplicación con nadie y no incluirla directamente en el código fuente de tu aplicación (*hardcodear*). En su lugar, es recomendable utilizar variables de entorno o un archivo de configuración para almacenar la contraseña de aplicación de forma segura.

2.1.0.2. ¿Cómo almacenar las credenciales de forma segura? En primer lugar es importante mencionar que, si utilizamos un fichero para almacenar las credenciales, hemos de asegurarnos de que este fichero **no se incluya en el control de versiones** (incluirlo en el fichero `.gitignore`) para evitar que se suban al repositorio y se expongan *a todo el mundo*. Por ejemplo, si utilizamos un archivo `.env` para almacenar las credenciales, debemos agregar la siguiente línea al archivo `.gitignore`:

```
.env
```

Otra opción es excluir todos los archivos en el fichero `.gitignore` e incluir solo los archivos que queremos subir al repositorio:

```
# Ignore everything
*

# But not these files...
!/.gitignore

!* .go
!go.sum
!go.mod

!README.md
!LICENSE
```

La idea de utilizar este fichero `.env` se debe a que en Go disponemos de un paquete llamado `godotenv` que permite cargar variables de entorno desde dicho archivo. Esto que facilita la gestión de credenciales y otros parámetros de configuración sin exponerlos en el código fuente. Para usar este método, simplemente hemos de crear un archivo `.env` en el mismo directorio que nuestro código fuente con el siguiente contenido:

```
VAR_1=valor_1
VAR_2=valor_2
```

Como podemos ver se trata de parejas clave-valor, donde cada línea representa una variable de entorno y su valor correspondiente. En este ejemplo, hemos definido dos variables de entorno: `VAR_1` con el valor `valor_1` y `VAR_2` con el valor `valor_2`. Podemos incluir tantas variables de entorno como necesitemos para configurar nuestra aplicación.

Para recuperar el valor de estas variables de entorno en nuestro código Go, hemos de invocar la función `godotenv.Load()` al inicio de nuestra función `main` para luego acceder a los valores de las variables de entorno, como cualquier otra variable de entorno, utilizando la función `os.Getenv` del paquete `os`. Por ejemplo, para obtener el valor de `VAR_1` y `VAR_2`, podemos hacer lo siguiente:

```
import (
    "log"
    "os"

    // Incluir el paquete `godotenv` en nuestro código Go para cargar las variables
    // de entorno desde el archivo `.env`:
    "github.com/joho/godotenv"
)
```

A continuación, podemos cargar las variables de entorno al inicio de nuestra función `main`:

```
func main() {
    // Cargar las variables de entorno desde el archivo .env
    err := godotenv.Load()
    if err != nil {
        log.Fatalf("Error loading .env file:%v", err)
    }

    // Ahora podemos acceder a las variables de entorno utilizando os.Getenv
    var1 := os.Getenv("VAR_1")
    var2 := os.Getenv("VAR_2")

    // Resto del código...
}
```

2.2. Envío de emails usando `net/smtp` con el servidor SMTP de Gmail

Ahora que hemos obtenido la contraseña de aplicación, podemos configurar el servidor SMTP de Gmail en nuestro código Go utilizando el paquete `net/smtp`. Para ello, necesitaremos especificar:

- La dirección del servidor SMTP de Gmail: `smtp.gmail.com`
- El puerto que se utilizará para la comunicación (587 para TLS o 465 para SSL).
- Las credenciales de autenticación:
 - La dirección de correo electrónico del remitente (nuestra cuenta de Gmail).
 - La contraseña de la aplicación (obtenida en el paso anterior).

```
package main

import (
    "bytes"
    "fmt"
    "log"
    "net/smtp"
    "os"
)

const (
    server      = "smtp.gmail.com"
    port        = 587
    appPassFile = "app.pass" // Nombre del archivo que contiene la contraseña
    ↵ de aplicación.
```

```
    crlf      = "\r\n"
)

func main() {
    // Hemos de obtener el password de la aplicación desde el entorno o, para
    ↪ este ejemplo, desde un fichero.
    appPass, err := os.ReadFile(appPassFile)
    if err != nil {
        // Si no disponemos del password, no podemos continuar.
        log.Fatalf("Error reading app password:%v", err)
    }

    // Ahora hemos de establecer los valores necesarios para la conexión con el
    ↪ servidor SMTP de Gmail.
    // El email del remitente, el destinatario, el asunto y el cuerpo del
    ↪ mensaje.
    from := "asincrono@gmail.com"
    to := "mmourazos@gmail.com"
    subject := "Correo de prueba desde Go"
    // El cuerpo del mensaje.
    body := "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do
    ↪ eiusmod tempor incididunt ut labore et dolore magna aliqua."

    // Creamos un buffer ya que al final lo que le hemos de pasar al servidor
    ↪ SMTP es una secuencia de bytes con el mensaje completo, incluyendo los
    ↪ encabezados.
    msg := bytes.Buffer{}
    fmt.Fprintf(&msg, "From:%s", from)
    msg.WriteString(crlf)
    fmt.Fprintf(&msg, "To:%s", to)
    msg.WriteString(crlf)
    fmt.Fprintf(&msg, "Subject:%s", subject)
    msg.WriteString(crlf)
    msg.WriteString(crlf) // Línea en blanco entre los encabezados y el cuerpo
    ↪ del mensaje.
    msg.WriteString(body)

    // Como en el caso del envío de mensajes http a través de un socket tcp, cada
    ↪ línea del mensaje debe terminar con un CRLF (carriage return + line feed)
    ↪ para que el servidor SMTP pueda interpretar correctamente el mensaje.
    // También valdría terminar en "\n" pero es recomendable usar '\r\n' para
    ↪ asegurar la compatibilidad con todos los servidores SMTP.

    address := fmt.Sprintf("%s:%d", server, port)
    err = smtp.SendMail(address, smtp.PlainAuth("", from, string(appPass),
    ↪ server), from, []string{to}, msg.Bytes())
}
```

```
if err != nil {  
    log.Printf("Error sending email:%v", err)  
}  
}
```

En el siguiente documento veremos cómo utilizar el paquete go-mail para realizar el envío de correos electrónicos de una forma más sencilla y con más funcionalidades que el paquete net/smtp. Este paquete es una biblioteca de terceros que proporciona una interfaz más amigable y fácil de usar para enviar correos electrónicos desde aplicaciones Go, permitiendo la configuración de encabezados, adjuntos, formatos de mensaje, entre otras características avanzadas.